Lesson 1: Introduction to Terraform

What is Terraform?

- **Terraform** is an open-source Infrastructure as Code (IaC) tool that allows you to define and provision data center infrastructure using a high-level configuration language known as HashiCorp Configuration Language (HCL).
- It enables users to automate the management of infrastructure such as servers, storage, networks, and more.

Why Use Terraform?

- **Declarative Syntax**: You define *what* infrastructure you need, not *how* to create it.
- **Infrastructure as Code**: It allows you to version control your infrastructure, just like code.
- **Platform Agnostic**: It supports major cloud platforms like AWS, Azure, Google Cloud, and on-premises infrastructure.

Key Concepts

- **Providers**: The cloud platforms or services Terraform interacts with (e.g., AWS, Azure).
- **Resources**: Define the components of your infrastructure (e.g., virtual machines, storage).
- **Modules**: Reusable sets of Terraform configuration files.
- **State**: Terraform tracks your infrastructure's state to determine what changes are needed.

PATHNEX
Shaping Careers, Building Futures

Important: Throughout these notes, HCL (HashiCorp Configuration Language) is used for defining infrastructure in Terraform configuration files, while Bash is used for running Terraform commands in the terminal. Make sure you understand the difference between the configuration syntax (HCL) and the commands you execute (Bash).

Lesson 2: Setting Up Terraform

Step 1: Install Terraform

- Visit the Terraform downloads page and follow the instructions to install Terraform for your operating system.
- After installation, verify by running:

```
Bash
terraform --version
```

Step 2: Configure a Provider

- Example: AWS Provider setup
  - To interact with AWS, you'll need to configure the AWS provider in your Terraform configuration.
  - You'll need AWS credentials (Access Key ID and Secret Access Key), either through the `AWS CLI` or environment variables.

Example `provider.tf` file:

```
hcl
provider "aws" {
  region = "us-west-2"
}
```

Lesson 3: Writing Terraform Configuration

Step 1: Define a Resource

- Terraform uses resources to define infrastructure components. Here's an example that creates an EC2 instance in AWS.

PATHNEX
Shaping Careers, Building Futures

Example `main.tf` file:

```
hcl
provider "aws" {
  region = "us-west-2"
}

resource "aws_instance" "my_ec2" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
}
```

- In the above example:
  - `aws_instance` is the resource type (EC2 instance).
  - `my_ec2` is the resource name (this can be anything).
  - `ami` specifies the AMI to use.
  - `instance_type` specifies the instance type.

Step 2: Initialize Terraform

- Before using Terraform to apply changes, run:

  ```
  bash
  terraform init
  ```

This command initializes your working directory and downloads the necessary provider plugins.

Lesson 4: Terraform Commands

1. `terraform plan`

- Terraform reads your configuration files and compares them with the current infrastructure state.
- It then shows what changes it will make.

```
bash
terraform plan
```

2. `terraform apply`

- This command actually makes the changes to your infrastructure.

```
bash
terraform apply
```

3. `terraform destroy`

- If you want to tear down everything that Terraform created:

```
bash
terraform destroy
```

4. `terraform validate`

- Validates the configuration files for syntax errors.

```
bash
terraform validate
```

Lesson 5: Variables and Outputs

Using Variables

- Terraform allows you to define variables to make your configuration more flexible and reusable.

Example `variables.tf` file:

```
hcl
variable "instance_type" {
  description = "The instance type"
```

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

```
  default     = "t2.micro"
}
```

- You can reference the variable inside the resource:

```
hcl
resource "aws_instance" "my_ec2" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = var.instance_type
}
```

Using Outputs

- Outputs allow you to display important information about the resources you've created.

Example `outputs.tf`:

```
hcl
output "instance_public_ip" {
  value = aws_instance.my_ec2.public_ip
}
```

Lesson 6: Terraform State

PATHNEX
Shaping Careers, Building Futures

What is Terraform State?

- Terraform maintains a state file (`terraform.tfstate`) to keep track of the resources it manages.
- Terraform uses this state to detect changes between the actual infrastructure and your configuration.

Important State Commands:

- `terraform state list`: Lists all resources in the state file.
- `terraform state show <resource>`: Shows the detailed state of a specific resource.

Remote State Storage

- For teams, it's common to store state remotely (e.g., in an S3 bucket) to avoid conflicts and ensure consistency.

Lesson 7: Modules

What Are Terraform Modules?

- A module is a container for multiple resources that are used together. It allows you to organize your Terraform code better and reuse it.

Creating a Module

- Example: A simple EC2 instance module might look like this:

1. Create a folder `modules/ec2-instance/` with `main.tf`:

```
hcl
resource "aws_instance" "my_instance" {
  ami           = var.ami
  instance_type = var.instance_type
}
```

2. Create a `variables.tf` file in the same folder:

```
hcl
variable "ami" {
  description = "AMI ID for EC2 instance"
}

variable "instance_type" {
  description = "EC2 instance type"
}
```

3. Use the module in the root configuration:

```
hcl
module "ec2_instance" {
  source        = "./modules/ec2-instance"
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
}
```

Lesson 8: Working with Remote Backends

Why Use Remote Backends?

- Remote backends store the Terraform state file remotely, making it easier for teams to collaborate.

Example `backend.tf`:

```
hcl
terraform {
  backend "s3" {
    bucket = "my-terraform-state-bucket"
```

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

```
    key    = "state/terraform.tfstate"
    region = "us-west-2"
  }
}
```

Lesson 9: Terraform Lifecycle (Notes)

📌 Definition

- In Terraform, the **`lifecycle` block** controls how resources are **created, updated, and destroyed**.
- Used inside a `resource` block.

📌 Syntax

```
resource "<provider>_<type>" "<name>" {
  # resource config

  lifecycle {
    # lifecycle rules
  }
}
```

PATHNEX
Shaping Careers, Building Futures

🔷 Lifecycle Arguments

1. `create_before_destroy`

- Creates new resource **before deleting old one**

```
lifecycle {
  create_before_destroy = true
}
```

✅ Use cases:

- Zero downtime deployments
- Load balancers / servers

⚠️ Notes:

- May increase cost temporarily
- Might fail if names must be unique

---

2. `prevent_destroy`

- Prevents accidental deletion

```
lifecycle {
  prevent_destroy = true
}
```

✅ Use cases:

- Production DB
- Critical infra

⚠️ Terraform will throw error on destroy

---

3. `ignore_changes`

- Ignores changes to selected attributes

```
lifecycle {
  ignore_changes = [tags]
}
```

✅ Use cases:

- External updates (e.g., autoscaling, tags)

👉 Ignore all:

```
ignore_changes = all
```

⚠️ Risk:

- Can hide configuration drift

---

4. `replace_triggered_by`

- Forces resource replacement when dependency changes

```
lifecycle {
  replace_triggered_by = [aws_ami.latest]
}
```

✅ Use cases:

- Rebuild EC2 when AMI changes

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

- Immutable infra

📊 Quick Summary Table

Argument	Purpose
`create_before_destroy`	Zero downtime replacement
`prevent_destroy`	Protect resource
`ignore_changes`	Ignore drift
`replace_triggered_by`	Force rebuild

⚠️ Important Points

- Works **only in resource block**
- Not applicable to modules
- Overusing `ignore_changes` is risky
- Provider limitations may apply

🧠 Memory Trick

👉 **C-P-I-R**

- **C** → Create before destroy
- **P** → Prevent destroy
- **I** → Ignore changes
- **R** → Replace triggered

PATHNEX
Shaping Careers, Building Futures

Lesson 10: Best Practices

1. **Keep Secrets Secure**: Never hard-code secrets like AWS credentials in the configuration files.
2. **Version Control**: Always version control your Terraform code but not the state files.
3. **Use `terraform fmt`**: Format your code consistently:

   ```
   bash
   terraform fmt
   ```

Hands-on Example Project

1. Create a project where students create an S3 bucket and EC2 instance in AWS using Terraform.
2. Configuring the provider, writing resource blocks, planning, and applying the changes.
3. Familiar with TF using variables to customize the configuration (e.g., instance types, regions).

PATHNEX
Shaping Careers, Building Futures